

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 9

дисциплина: Архитектура компьютера

Студент: Ксения Танчила

Группа: НКАбд-05-25

МОСКВА

2025 г.

Оглавление

Цель работы.....	3
Задание.....	4
Теоритическое введение.....	5
Выполнение лабораторной работы.....	6
Реализация подпрограмм в NASM.....	6
Отладка программ с помощью GDB.....	9
Добавление точек останова.....	12
Работа с данными программ GDB.....	14
Обработка аргументов командной строки в GDB.....	21
Задания для самостоятельной работы.....	24
Выводы.....	27
Список литературы.....	28

1 Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Задание

1. Реализация подпрограмм в NASM
2. Отладка программ с помощью GDB
3. Самостоятельное выполнение заданий по материалам лабораторной работы

3 Теоретическое введение

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа:

- обнаружение ошибки;
- поиск её местонахождения;
- определение причины ошибки;
- исправление ошибки.

Можно выделить следующие типы ошибок:

- синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка;
- семантические ошибки — являются логическими и приводят к тому, что программа запускается, отработывает, но не даёт желаемого результата;
- ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль).

Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга.

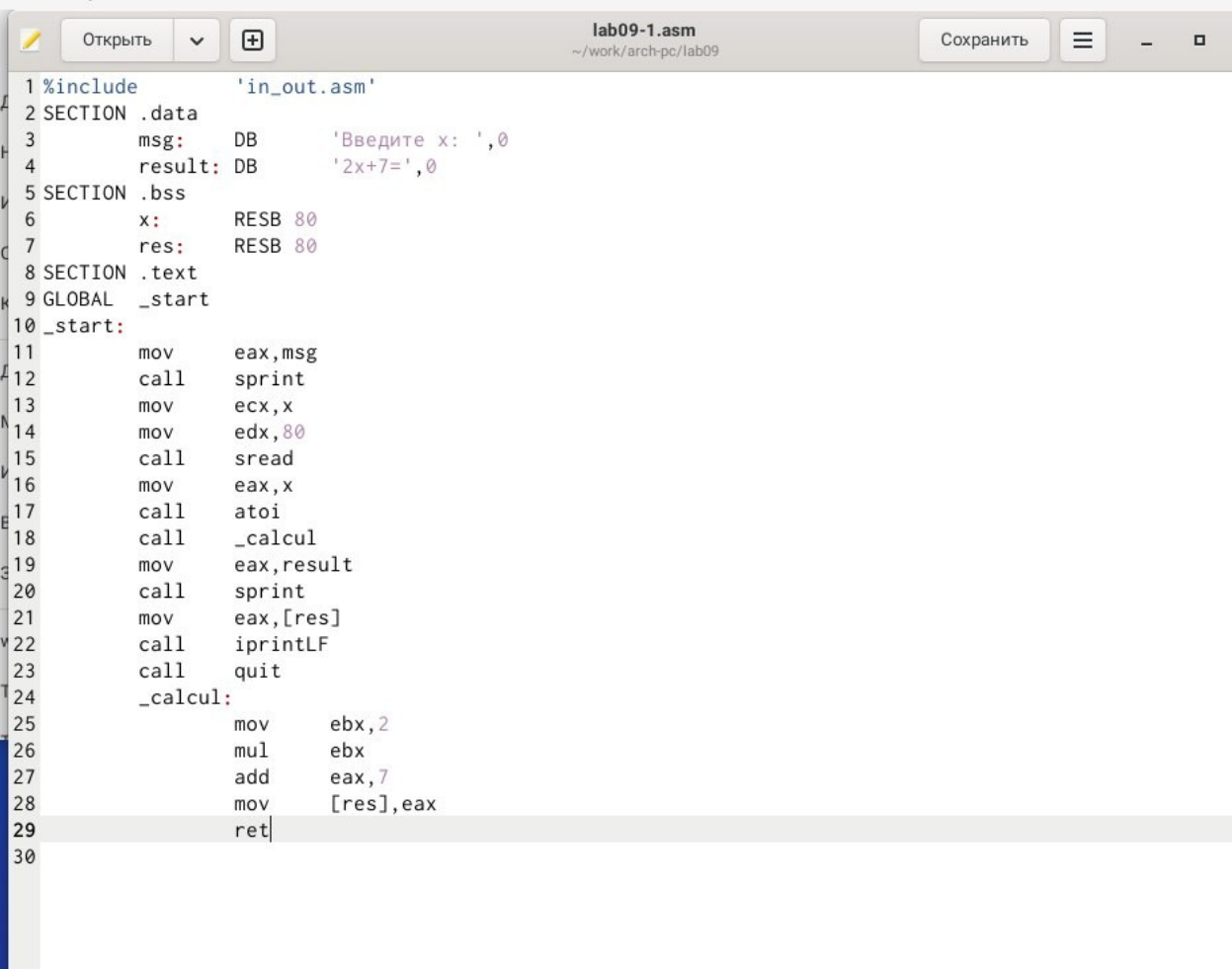
Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы. Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

4 Выполнение лабораторной работы

4.1 Реализация подпрограмм в NASM

1. Создала каталог для программам лабораторной работы № 9, перешла в него и создала файл lab9-1.asm.

2. В качестве примера рассмотрели программу вычисления арифметического выражения $f(x) = 2x + 7$ с помощью подпрограммы `_calcul`. В данном примере x вводится с клавиатуры, а само выражение вычисляется в подпрограмме. Внимательно изучила текст программы Листинга 9.1 и ввела его в файл lab9-1.asm (Рис. 4.1.1).

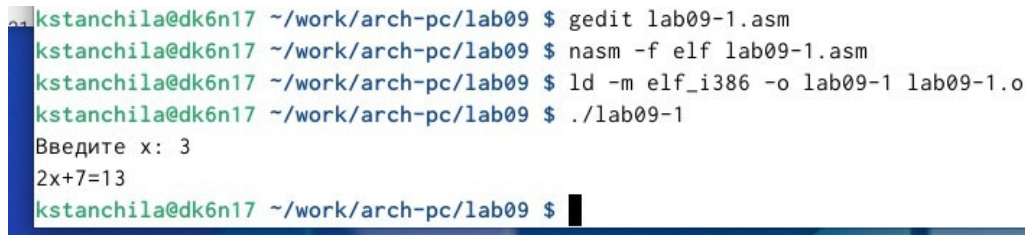


```
1 %include 'in_out.asm'
2 SECTION .data
3     msg: DB 'Введите x: ',0
4     result: DB '2x+7=',0
5 SECTION .bss
6     x: RESB 80
7     res: RESB 80
8 SECTION .text
9 GLOBAL _start
10 _start:
11     mov     eax,msg
12     call    sprint
13     mov     ecx,x
14     mov     edx,80
15     call    sread
16     mov     eax,x
17     call    atoi
18     call    _calcul
19     mov     eax,result
20     call    sprint
21     mov     eax,[res]
22     call    iprintLF
23     call    quit
24     _calcul:
25         mov     ebx,2
26         mul     ebx
27         add     eax,7
28         mov     [res],eax
29         ret
30
```

Рис. 4.1.1 — Текст программы lab9-1.asm

Первые строки программы отвечают за вывод сообщения на экран (`call sprint`), чтение данных введенных с клавиатуры (`call sread`) и преобразования

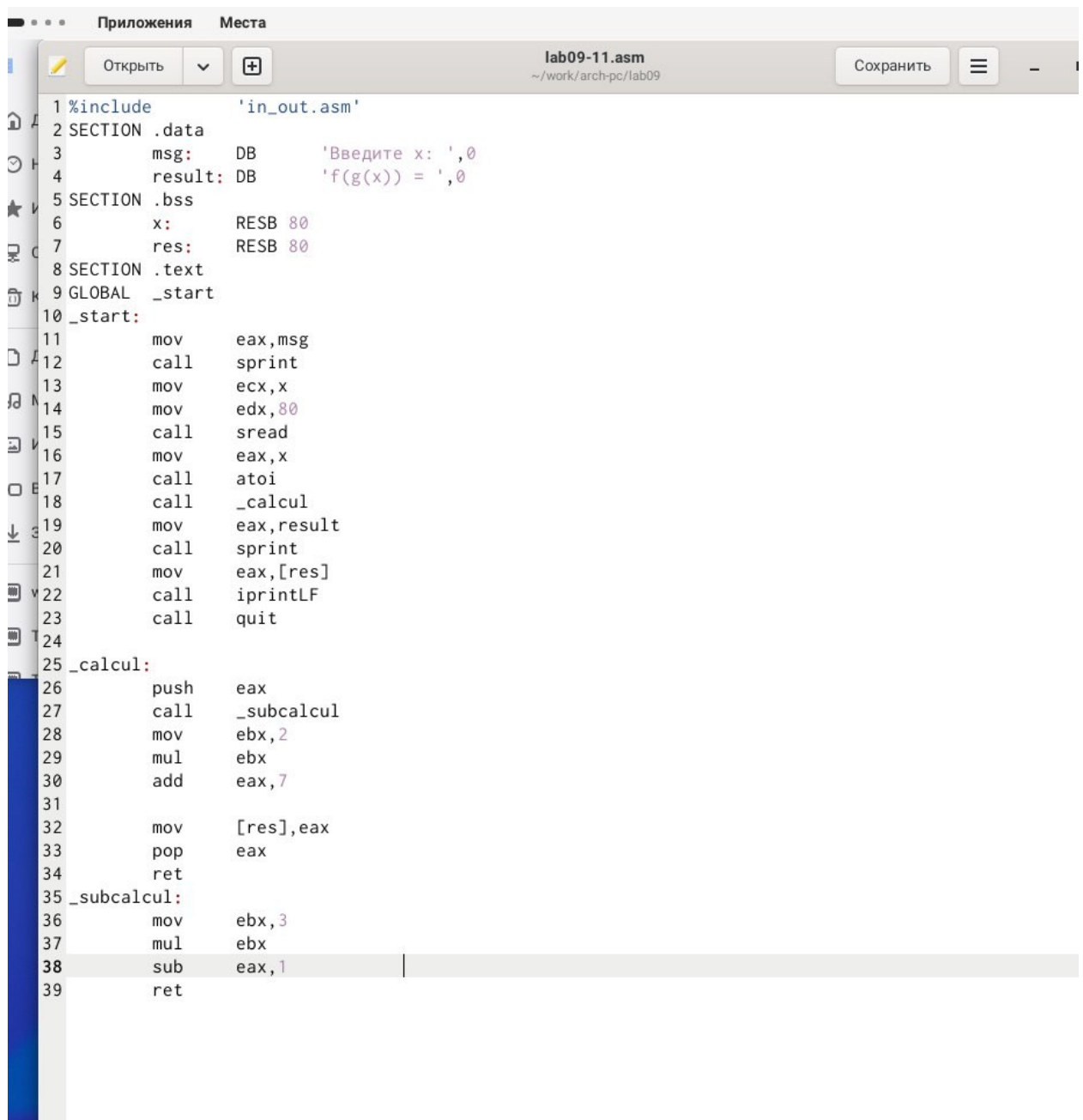
введенных данных из символьного вида в численный (call atoi). Инструкция get является последней в подпрограмме и ее исполнение приводит к возвращению в основную программу к инструкции, следующей за инструкцией call, которая вызвала данную подпрограмму. Последние строки программы реализуют вывод сообщения (call sprint), результата вычисления (call iprintLF) и завершение программы (call quit). Создала исполняемый файл и проверила его работу (Рис. 4.1.2).



```
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ gedit lab09-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ nasm -f elf lab09-1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-1 lab09-1.o
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ./lab09-1
Введите x: 3
2x+7=13
kstanchila@dk6n17 ~/work/arch-pc/lab09 $
```

Рис. 4.1.2 — Вывод программы lab9-1.asm

Изменила текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$. Т.е. x передается в подпрограмму `_calcul` из нее в подпрограмму `_subcalcul`, где вычисляется выражение $g(x)$, результат возвращается в `_calcul` и вычисляется выражение $f(g(x))$. Результат возвращается в основную программу для вывода результата на экран. Текст программы представлен на Рис. 4.1.3. Создала исполняемый файл и проверила его работу (Рис. 4.1.4).



```
1 %include 'in_out.asm'
2 SECTION .data
3     msg: DB 'Введите x: ',0
4     result: DB 'f(g(x)) = ',0
5 SECTION .bss
6     x: RESB 80
7     res: RESB 80
8 SECTION .text
9 GLOBAL _start
10 _start:
11     mov     eax,msg
12     call    sprintf
13     mov     ecx,x
14     mov     edx,80
15     call    sread
16     mov     eax,x
17     call    atoi
18     call    _calcul
19     mov     eax,result
20     call    sprintf
21     mov     eax,[res]
22     call    iprintLF
23     call    quit
24
25 _calcul:
26     push    eax
27     call    _subcalcul
28     mov     ebx,2
29     mul     ebx
30     add     eax,7
31
32     mov     [res],eax
33     pop     eax
34     ret
35 _subcalcul:
36     mov     ebx,3
37     mul     ebx
38     sub     eax,1
39     ret
```

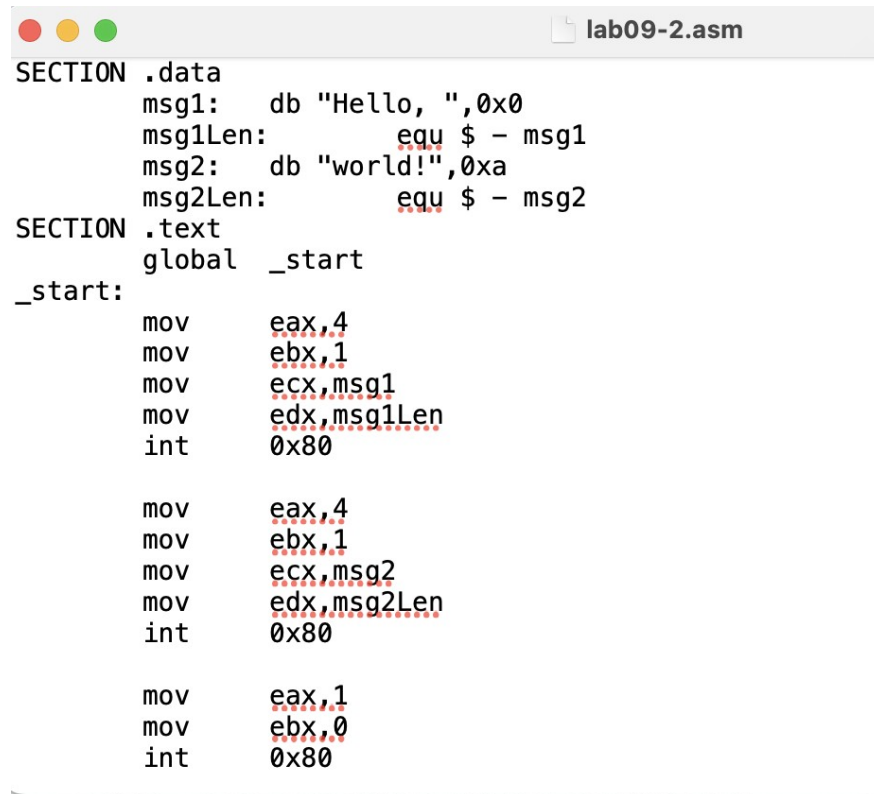
Рис. 4.1.3 — Текст программы lab9-11.asm

```
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ touch lab09-11.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ gedit lab09-11.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ nasm -f elf lab09-11.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-11 lab09-11.o
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ./lab09-11
Введите x: 3
f(g(x)) = 23
kstanchila@dk6n17 ~/work/arch-pc/lab09 $
```

Рис. 4.1.4 — Вывод программы lab9-11.asm

4.2 Отладка программ с помощью GDB

3. Создала файл lab09-2.asm с текстом программы из Листинга 9.2. (Программа печати сообщения Hello world!): (Рис. 4.2.1).



```
SECTION .data
msg1:  db "Hello, ",0x0
msg1Len:  equ $ - msg1
msg2:  db "world!",0xa
msg2Len:  equ $ - msg2

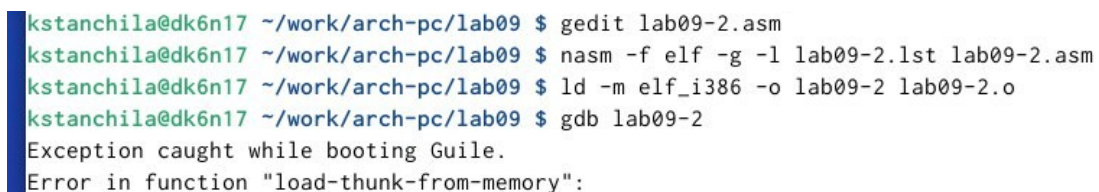
SECTION .text
global _start
_start:
    mov     eax,4
    mov     ebx,1
    mov     ecx,msg1
    mov     edx,msg1Len
    int     0x80

    mov     eax,4
    mov     ebx,1
    mov     ecx,msg2
    mov     edx,msg2Len
    int     0x80

    mov     eax,1
    mov     ebx,0
    int     0x80
```

Рис. 4.2.1 — Текст листинга 9.2

Получила исполняемый файл. Для работы с GDB в исполняемый файл необходимо добавить отладочную информацию, для этого трансляцию программ необходимо проводить с ключом ‘-g’. (Рис. 4.2.2).



```
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ gedit lab09-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-2.lst lab09-2.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-2 lab09-2.o
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ gdb lab09-2
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
```

Рис. 4.2.2 — Компиляция программы lab9-2.asm

Загрузила исполняемый файл в отладчик gdb.

Проверила работу программы, запустив ее в оболочке GDB с помощью команды run (сокращённо r) (Рис. 4.2.3).

```
кстanchила@dk6n17 - lab09
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/k/s/kstanchila/work/arch-pc/lab09/lab09-2
Hello, world!
[Inferior 1 (process 7361) exited normally]
```

Рис. 4.2.3 — run в оболочке GDB

Для более подробного анализа программы установила брейкпоинт на метку `_start`, с которой начинается выполнение любой ассемблерной программы, и запустила её. (Рис. 4.2.4).

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/k/s/kstanchila/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9      mov     eax,4
(gdb)
```

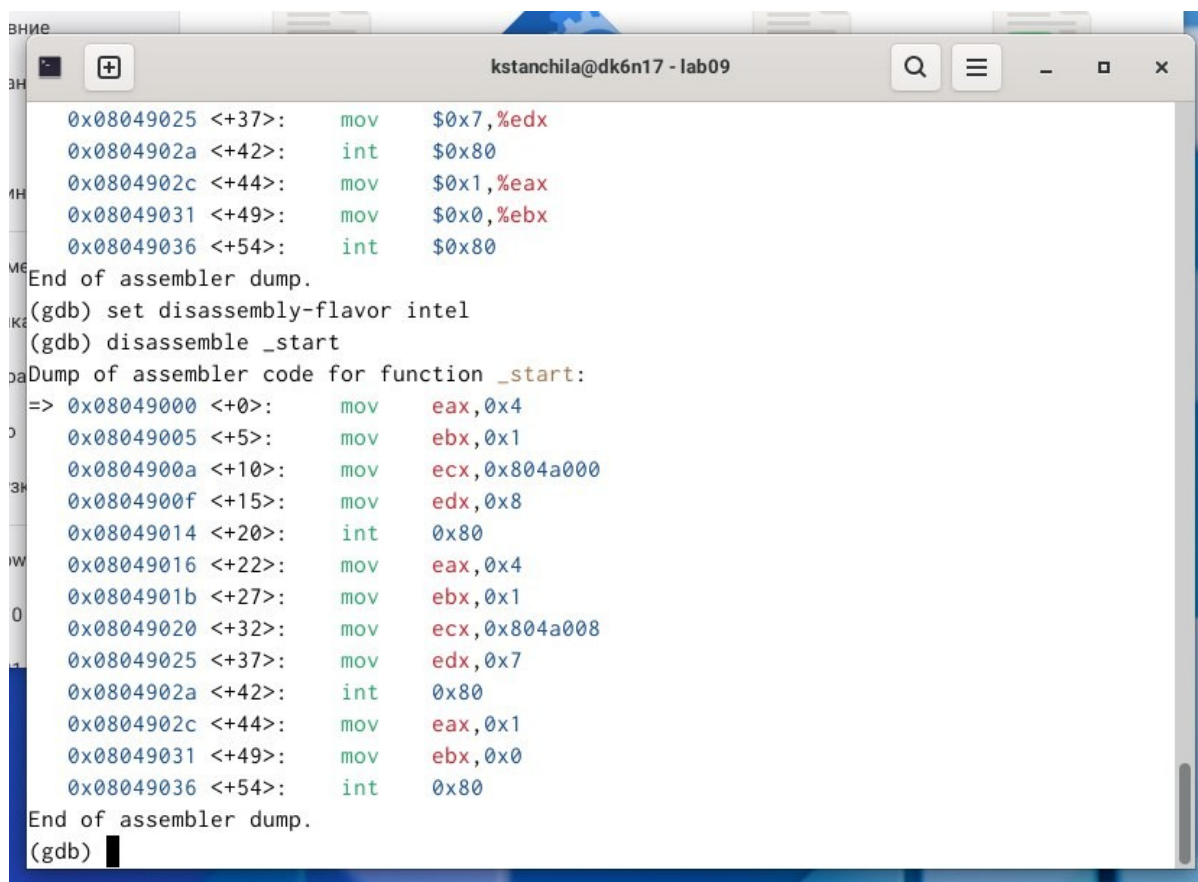
Рис. 4.2.4 — Брейкпоинт

Посмотрела дисассимилированный код программы с помощью команды `disassemble`, начиная с метки `_start` (Рис. 4.2.5).

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
0x08049005 <+5>:      mov     $0x1,%ebx
0x0804900a <+10>:     mov     $0x804a000,%ecx
0x0804900f <+15>:     mov     $0x8,%edx
0x08049014 <+20>:     int     $0x80
0x08049016 <+22>:     mov     $0x4,%eax
0x0804901b <+27>:     mov     $0x1,%ebx
0x08049020 <+32>:     mov     $0x804a008,%ecx
0x08049025 <+37>:     mov     $0x7,%edx
0x0804902a <+42>:     int     $0x80
0x0804902c <+44>:     mov     $0x1,%eax
0x08049031 <+49>:     mov     $0x0,%ebx
0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb)
```

Рис. 4.2.5 — disassemble

Переключитесь на отображение команд с Intel'овским синтаксисом, введя команду `set disassembly-flavor intel` (Рис. 4.2.6).



The screenshot shows a GDB terminal window with the title bar "kstanchila@dk6n17 - lab09". The terminal displays assembly code for a function named `_start`. The code is shown in Intel syntax, with instructions like `mov $0x7,%edx` and `int $0x80`. The addresses of the instructions are shown in hexadecimal, such as `0x08049025` and `0x0804902a`. The terminal also shows the command `(gdb) set disassembly-flavor intel` and `(gdb) disassemble _start` being entered. The output of the disassembly is shown, with instructions like `mov eax,0x4` and `mov ebx,0x1`. The terminal ends with `End of assembler dump.` and `(gdb)`.

```
0x08049025 <+37>:  mov    $0x7,%edx
0x0804902a <+42>:  int     $0x80
0x0804902c <+44>:  mov     $0x1,%eax
0x08049031 <+49>:  mov     $0x0,%ebx
0x08049036 <+54>:  int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a008
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.
(gdb)
```

Рис. 4.2.6 — `set disassembly-flavor intel`

Включила режим псевдографики для более удобного анализа программы (Рис. 4.2.7).

Различия отображения синтаксиса машинных команд в режимах АТТ и Intel:

1. Порядок операндов наоборот:
 - АТ&Т: ``src, dest`` (откуда → куда)
 - Intel: ``dest, src`` (куда ← откуда)
2. Разные обозначения:
 - АТ&Т: ``%reg``, ``$num``, ``(%eax)``
 - Intel: ``reg``, ``num``, ``[eax]``
3. Указание размера:
 - АТ&Т: в конце команды (``movb``, ``movw``)
 - Intel: перед операндом (``byte ptr``, ``word ptr``)

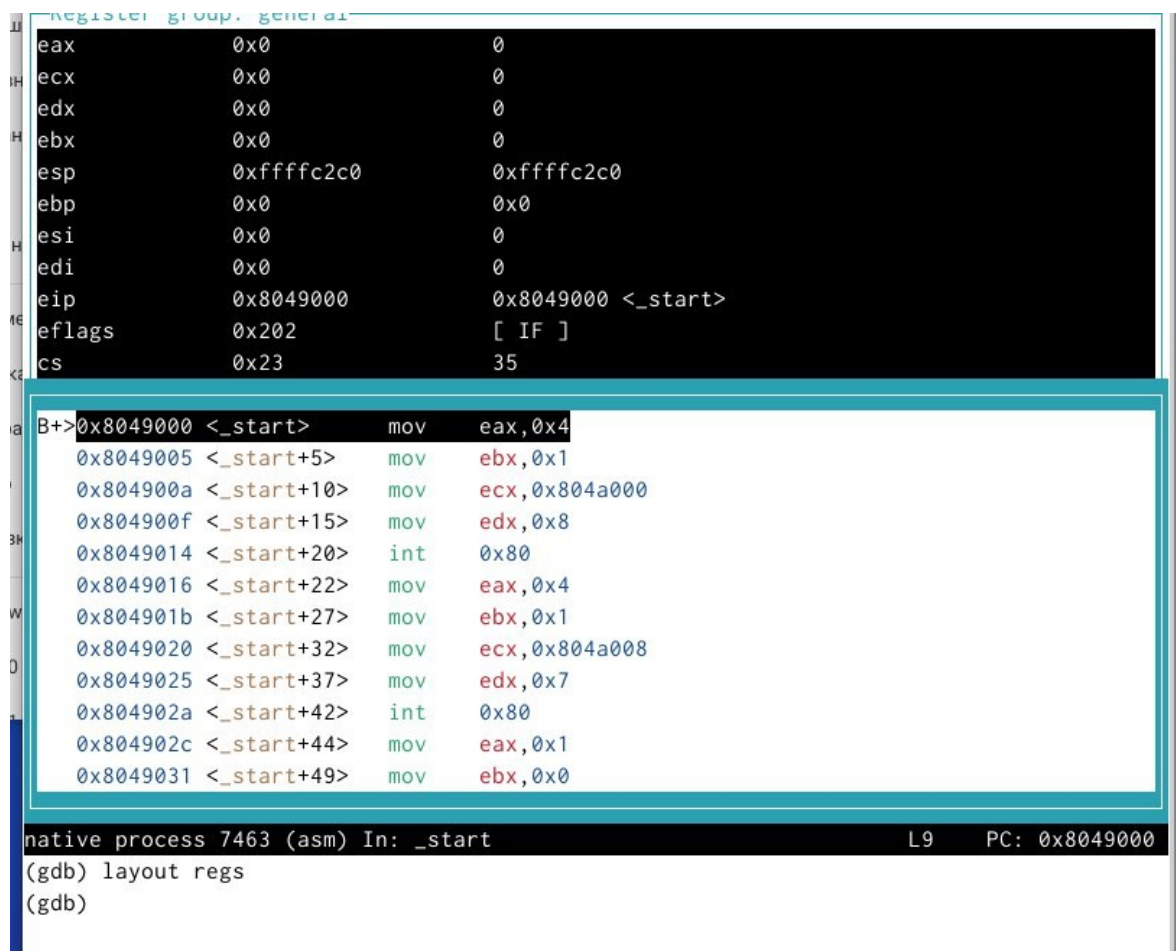


Рис. 4.2.7 — Режим псевдографики

В этом режиме есть три окна:

- В верхней части видны названия регистров и их текущие значения;
- В средней части виден результат дисассимилирования программы;
- Нижняя часть доступна для ввода команд.

4.2.1. Добавление точек останова

На предыдущих шагах была установлена точка останова по имени метки (`_start`). Проверила это с помощью команды `info breakpoints` (кратко `i b`) (Рис. 4.2.8):

The screenshot shows a GDB terminal window with the title bar 'kstanchila@dk7n07 - lab09'. The window is divided into three main sections. The top section, titled 'Register group: general', displays the values of general-purpose registers: `eax` (0x0), `ecx` (0x0), `edx` (0x0), `ebx` (0x0), `esp` (0xffffc2f0), and `ebp` (0x0). The middle section shows a list of assembly instructions with their addresses and disassembled forms: `0x804901b <_start+27> mov ebx,0x1`, `0x8049020 <_start+32> mov ecx,0x804a008`, `0x8049025 <_start+37> mov edx,0x7`, `0x804902a <_start+42> int 0x80`, `0x804902c <_start+44> mov eax,0x1`, and `0x8049031 <_start+49> mov ebx,0x0`. The bottom section shows the GDB prompt with the command `layout regs` and `info breakpoints`. The `info breakpoints` command output shows a single breakpoint at address `0x08049000` (labeled `lab09-2.asm:9`), which has been hit 1 time. The GDB prompt is currently at `(gdb) █`.

```
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffc2f0 0xffffc2f0
ebp      0x0      0x0

0x804901b <_start+27> mov ebx,0x1
0x8049020 <_start+32> mov ecx,0x804a008
0x8049025 <_start+37> mov edx,0x7
0x804902a <_start+42> int 0x80
0x804902c <_start+44> mov eax,0x1
0x8049031 <_start+49> mov ebx,0x0

native process 3518 (asm) In: _start L9 PC: 0x8049000
(gdb) layout regs
(gdb) info breakpoints
Num      Type          Disp Enb Address      What
1        breakpoint    keep y 0x08049000 lab09-2.asm:9
        breakpoint already hit 1 time
(gdb) █
```

Рис. 4.2.8 — info breakpoints

Установили еще одну точку останова по адресу инструкции. Адрес инструкции можно увидеть в средней части экрана в левом столбце соответствующей инструкции. Определила адрес предпоследней инструкции (`mov ebx,0x0`) и установила точку останова.

Посмотрела информацию о всех установленных точках останова (Рис. 4.2.9):

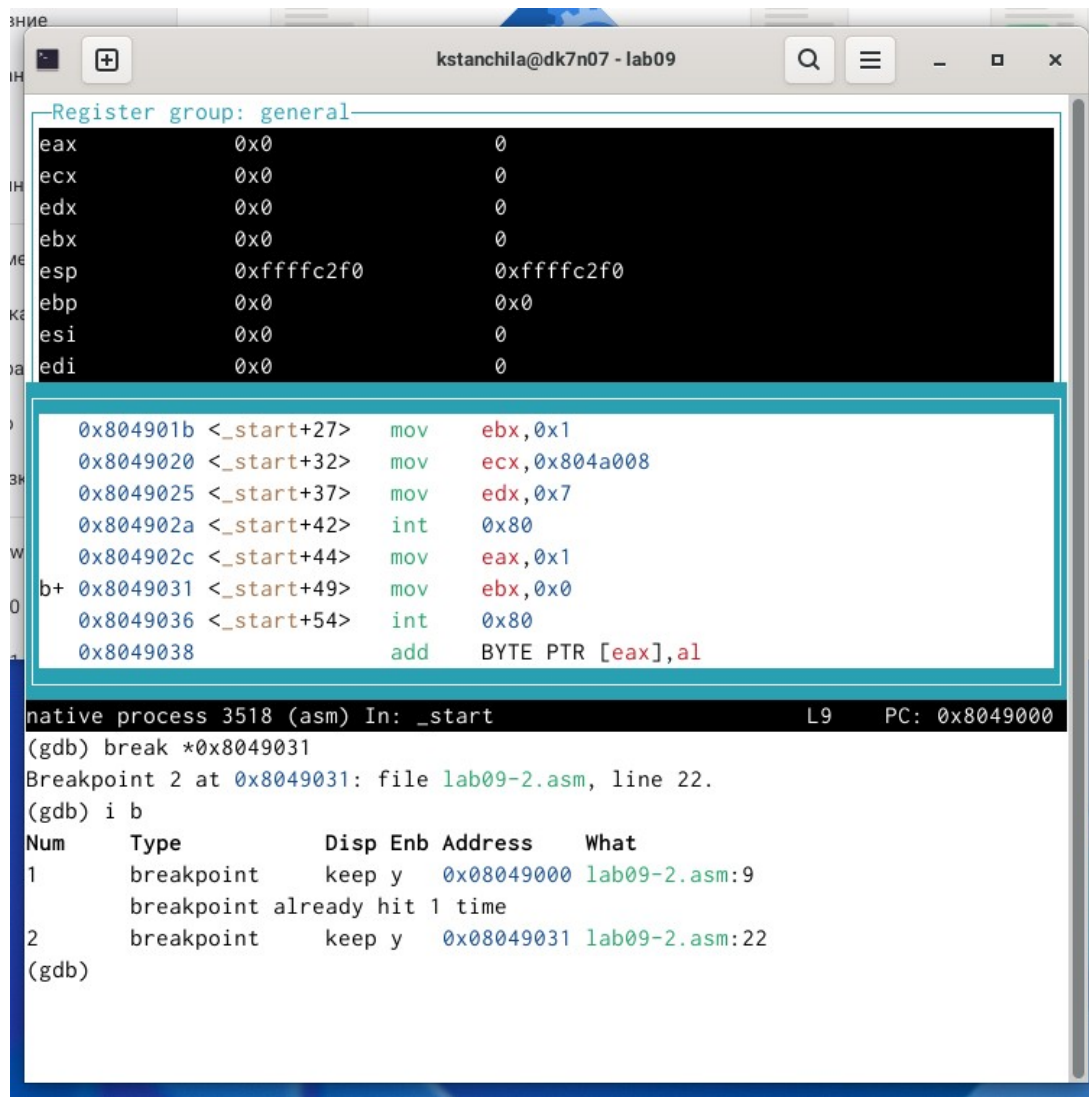


Рис. 4.2.9 — Точка останова

4.2.2. Работа с данными программы в GDB

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных. Выполнила 5 инструкций с помощью команды `stepi` (или `si`) и проследила за изменением значений регистров (Рис. 4.2.10).

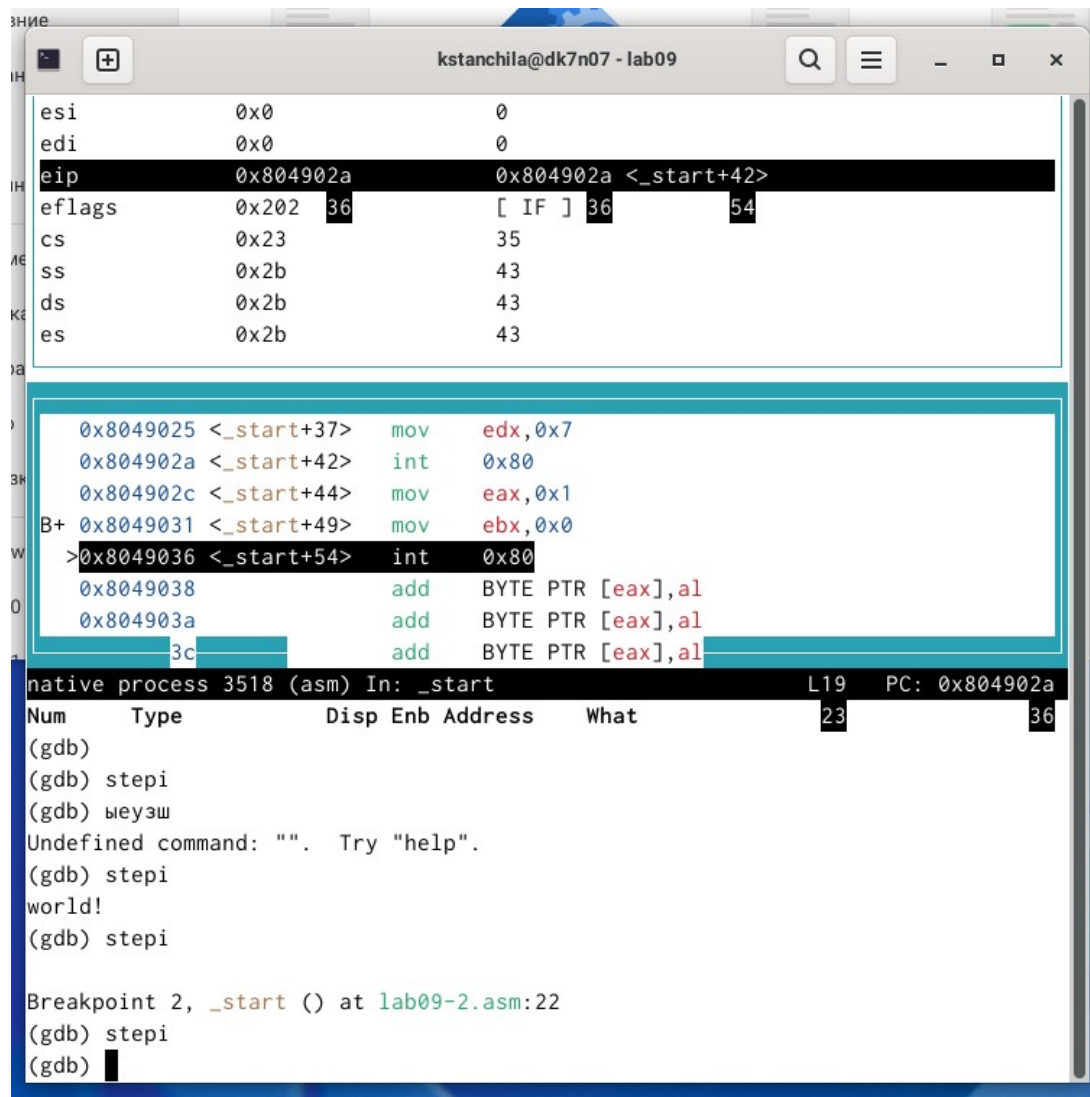


Рис. 4.2.10 — stepi

Посмотрела содержимое регистров с помощью команды `info registers`.

Для отображения содержимого памяти можно использовать команду `x` <адрес>, которая выдаёт содержимое ячейки памяти по указанному адресу. Формат, в котором выводятся данные, можно задать после имени команды через косую черту: `x/NFU <адрес>`. С помощью команды `x &<имя переменной>` также можно посмотреть содержимое переменной. Посмотрела значение переменной `msg1` по имени (Рис. 4.2.11).

```

kstanchila@dk7n07 - lab09

esi      0x0      0
edi      0x0      0
eip      0x804902a 0x804902a <_start+42>
eflags   0x202    36 [ IF ] 36 54
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43

0x8049025 <_start+37> mov    edx,0x7
0x804902a <_start+42> int    0x80
0x804902c <_start+44> mov    eax,0x1
B+ 0x8049031 <_start+49> mov    ebx,0x0
>0x8049036 <_start+54> int    0x80
0x8049038      add    BYTE PTR [eax],al
0x804903a      add    BYTE PTR [eax],al
3c      add    BYTE PTR [eax],al

native process 3518 (asm) In: _start L19 PC: 0x804902a
Num  Type      Disp Enb Address  What
esp      0xffffc2f0 0xffffc2f0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049036 0x8049036 <_start+54>
eflags   0x202    [ IF ]
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) x/1sb &msg1
0x804a000 <msg1>: "Hello, "
(gdb)

```

Рис. 4.2.11 — Значение переменной msg1

Посмотрела значение переменной msg2 по адресу. Адрес переменной можно определить по дизассемблированной инструкции. Посмотрела инструкцию mov esx,msg2 которая записывает в регистр esx адрес переменной msg2 (Рис. 4.2.12).


```

kstanchila@dk7n07 - lab09
esi      0x0      0
edi      0x0      0
eip      0x804902a 0x804902a <_start+42>
eflags   0x202    36 [ IF ] 36 54
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43

0x8049025 <_start+37> mov    edx,0x7
0x804902a <_start+42> int    0x80
0x804902c <_start+44> mov    eax,0x1
B+ 0x8049031 <_start+49> mov    ebx,0x0
>0x8049036 <_start+54> int    0x80
0x8049038          add    BYTE PTR [eax],al
0x804903a          add    BYTE PTR [eax],al
0x804903c          add    BYTE PTR [eax],al

native process 3518 (asm) In: _start L19 PC: 0x804902a
Num   Type   Disp Enb Address   What
esi    0x0      0
edi    0x0      0
eip    0x8049036 0x8049036 <_start+54>
eflags 0x202     [ IF ]
--Type <RET> for more, q to quit, c to continue without paging--q
Quit
(gdb) x/1sb &msg1
0x804a000 <msg1>: "Hello, "
(gdb) x/1sb 0x804a008
0x804a008 <msg2>: "world!\n\034"
(gdb)

```

Рис. 4.2.12 — mov ecx,msg2

Изменила значение для регистра или ячейки памяти можно с помощью команды `set`, задав ей в качестве аргумента имя регистра или адрес. При этом перед именем регистра ставится префикс `$`, а перед адресом нужно указать в фигурных скобках тип данных (размер сохраняемого значения; в качестве типа данных можно использовать типы языка Си). Изменила первый символ переменной `msg1` (Рис. 4.2.13).

The screenshot shows a GDB debugger window with the following content:

Register Window:

esi	0x0	0
edi	0x0	0
eip	0x804902a	0x804902a <_start+42>
eflags	0x202	[IF] 36 54
cs	0x23	35
ss	0x2b	43
ds	0x2b	43
es	0x2b	43

Assembly Window:

```

0x8049025 <_start+37>  mov    edx,0x7
0x804902a <_start+42>  int     0x80
0x804902c <_start+44>  mov    eax,0x1
B+ 0x8049031 <_start+49>  mov    ebx,0x0
>0x8049036 <_start+54>  int     0x80
0x8049038             add     BYTE PTR [eax],al
0x804903a             add     BYTE PTR [eax],al
0x804903c             add     BYTE PTR [eax],al

```

Command Window:

```

native process 3518 (asm) In: _start L19 PC: 0x804902a
Num   Type      Disp Enb Address  What
23    36
Symbol "msg1" is at 0x804a000 in a file compiled without debugging.
(gdb) set {char}msg1='h'
'msg1' has unknown type; cast it to its declared type
(gdb) whatis msg1
type = <data variable, no debug info>
(gdb) ptype msg1
type = <data variable, no debug info>
(gdb) set *(char*)&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "hello, "
(gdb)

```

Рис. 4.2.13 — Изменение первого символа переменной msg1

Заменяла символ во второй переменной msg2 (Рис. 4.2.14).

```

kstanchila@dk7n07 - lab09

Register Window:
esi      0x0      0
edi      0x0      0
eip      0x804902a 0x804902a <_start+42>
eflags   0x202    36 [ IF ] 36 54
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43

Assembly Window:
0x8049025 <_start+37> mov    edx,0x7
0x804902a <_start+42> int    0x80
0x804902c <_start+44> mov    eax,0x1
B+ 0x8049031 <_start+49> mov    ebx,0x0
>0x8049036 <_start+54> int    0x80
0x8049038          add    BYTE PTR [eax],al
0x804903a          add    BYTE PTR [eax],al
0x804903c          add    BYTE PTR [eax],al

Console Window:
native process 3518 (asm) In: _start L19 PC: 0x804902a
Num   Type      Disp Enb Address  What  23  36
(gdb) what is msg1
type = <data variable, no debug info>
(gdb) ptype msg1
type = <data variable, no debug info>
(gdb) set *(char*)&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "hello, "
(gdb) set *(char*)&msg2='K'
(gdb) x/1sb &msg2
0x804a008 <msg2>:      "Kor1d!\n\034"
(gdb)

```

Рис. 4.2.14 — Изменение символа переменной msg2

Вывела в различных форматах (в шестнадцатеричном формате, в двоичном формате и в символьном виде) значение регистра edx (Рис. 4.2.15).

```

kstanchila@dk7n07 - lab09
esi      0x0      0
edi      0x0      0
eip      0x804902a 0x804902a <_start+42>
eflags   0x202    36 [ IF ] 36 54
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43

0x8049025 <_start+37> mov    edx,0x7
0x804902a <_start+42> int    0x80
0x804902c <_start+44> mov    eax,0x1
B+ 0x8049031 <_start+49> mov    ebx,0x0
>0x8049036 <_start+54> int    0x80
0x8049038      add    BYTE PTR [eax],al
0x804903a      add    BYTE PTR [eax],al
0x804903c      add    BYTE PTR [eax],al

native process 3518 (asm) In: _start L19 PC: 0x804902a
Num   Type      Disp Enb Address  What
23    36
(gdb) x/1sb &msg2
0x804a008 <msg2>:      "Korld!\n\034"
(gdb) p/F $edx
No symbol "F" in current context.
(gdb) print/x $edx
$1 = 0x7
(gdb) print/t $edx
$2 = 111
(gdb) print/c $edx
$3 = 7 '\a'
(gdb)

```

Рис. 4.2.15 — Значение регистра edx

С помощью команды set изменила значение регистра ebx (Рис.4.2.16):

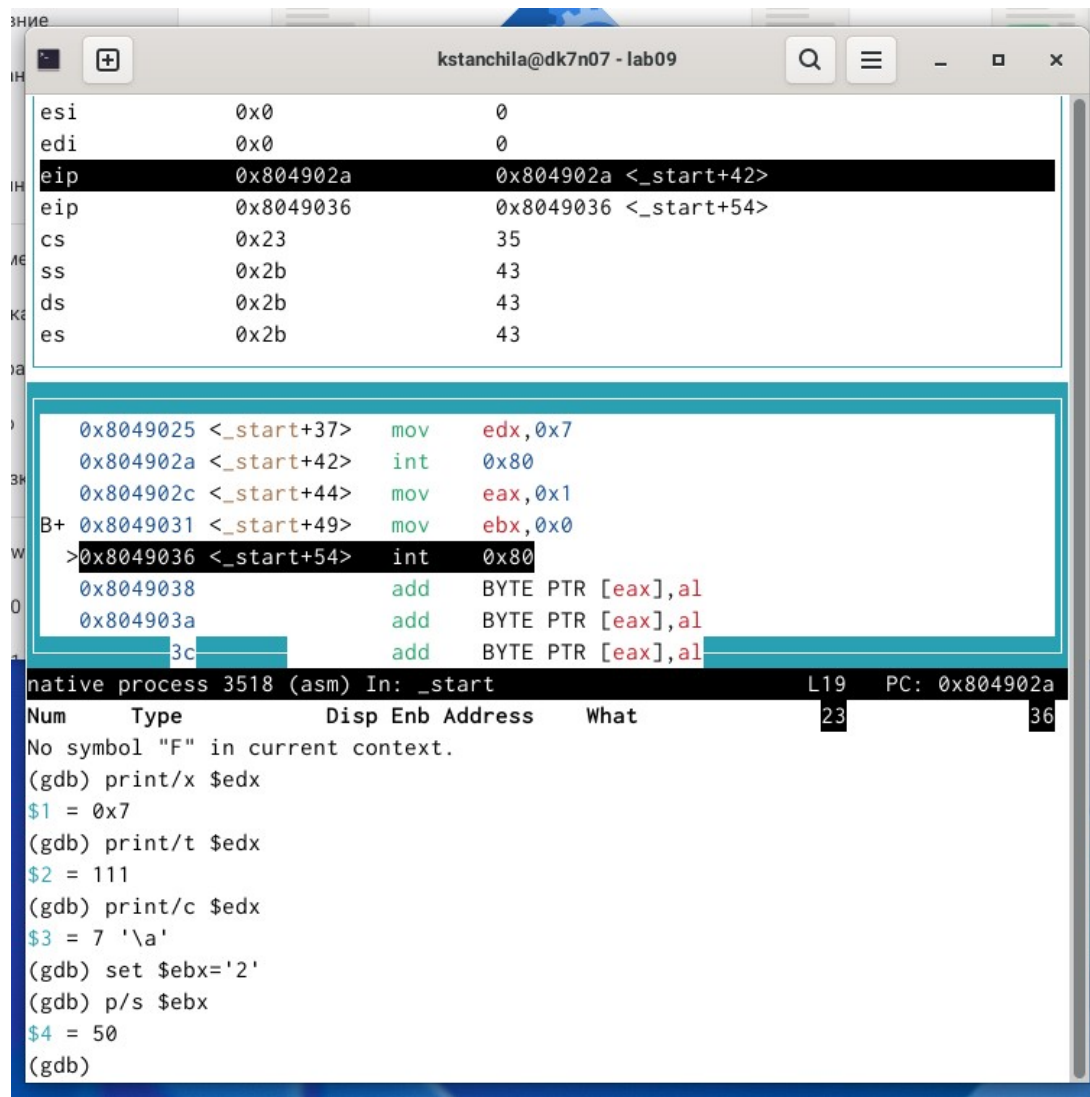


Рис. 4.2.16 — Изменение значения регистра ebx

Завершила выполнение программы с помощью команды `continue` (сокращенно `c`) или `stepi` (сокращенно `si`) и вышла из GDB с помощью команды `quit` (сокращенно `q`).

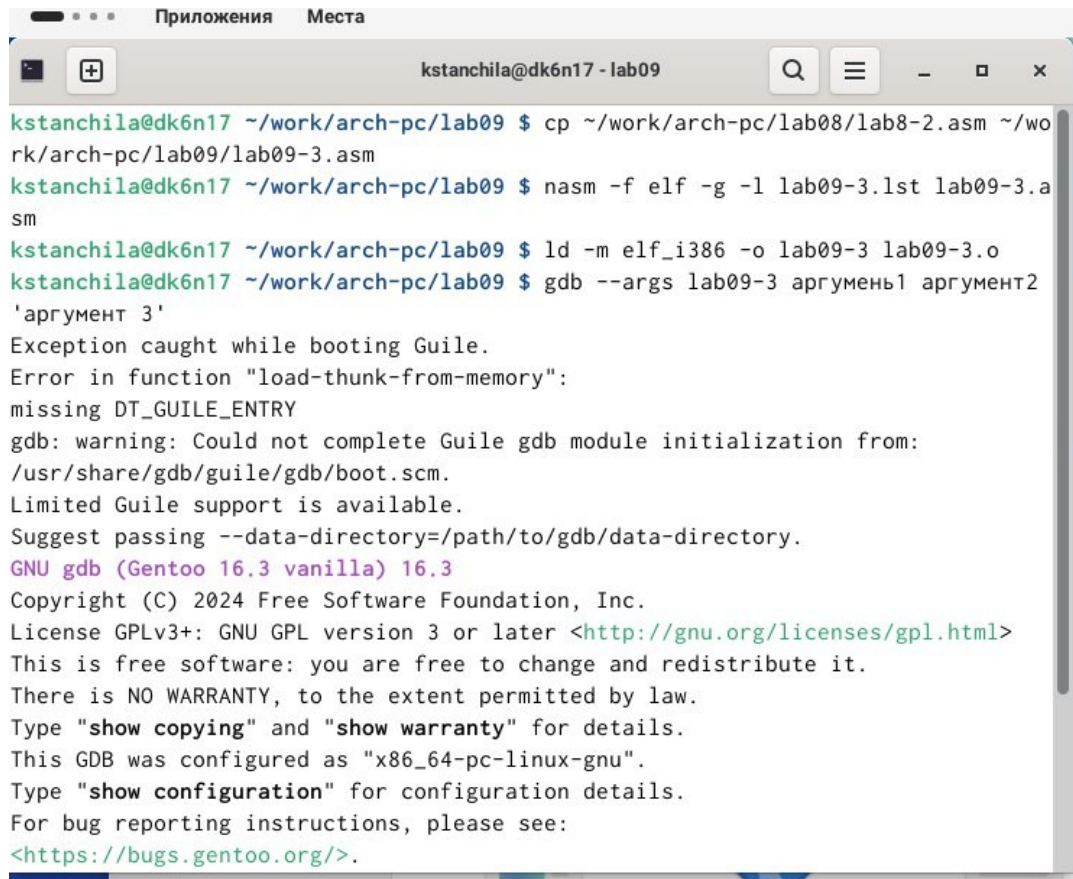
4.2.3. Обработка аргументов командной строки в GDB

Скопировала файл `lab8-2.asm`, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем `lab09-3.asm`.

Создала исполняемый файл.

Для загрузки в `gdb` программы с аргументами использовала ключ `—args`.

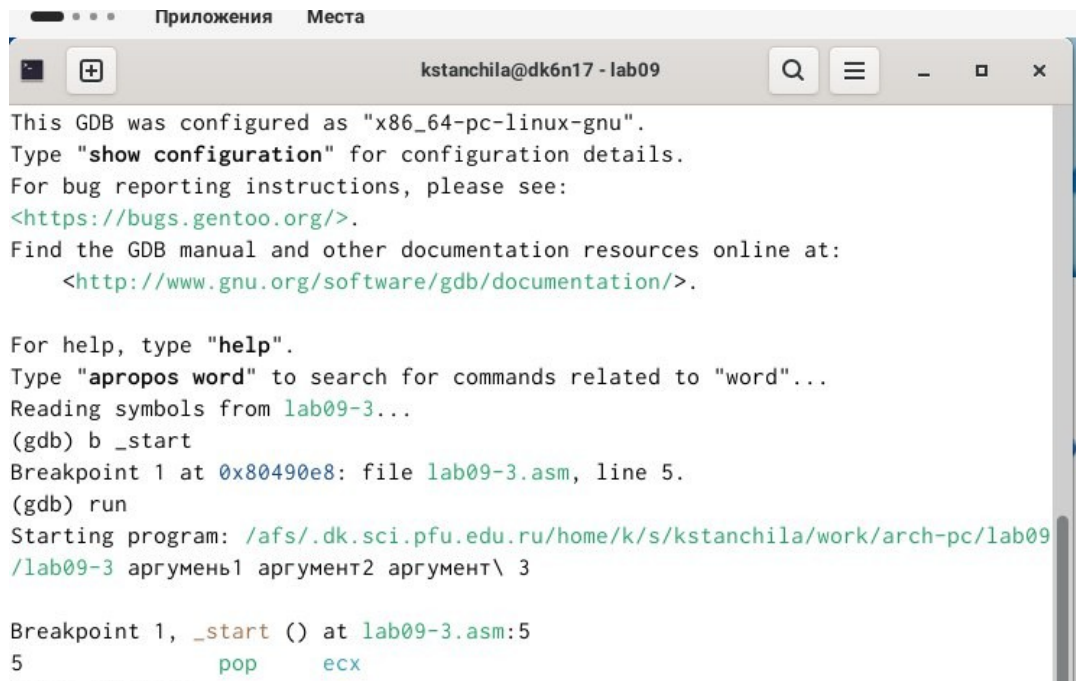
Загрузила исполняемый файл в отладчик, указав аргументы (Рис. 4.2.17):



```
kstanchila@dk6n17 - lab09
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ nasm -f elf -g -l lab09-3.lst lab09-3.asm
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ ld -m elf_i386 -o lab09-3 lab09-3.o
kstanchila@dk6n17 ~/work/arch-pc/lab09 $ gdb --args lab09-3 аргумент1 аргумент2 'аргумент 3'
Exception caught while booting Guile.
Error in function "load-thunk-from-memory":
missing DT_GUILLE_ENTRY
gdb: warning: Could not complete Guile gdb module initialization from:
/usr/share/gdb/guile/gdb/boot.scm.
Limited Guile support is available.
Suggest passing --data-directory=/path/to/gdb/data-directory.
GNU gdb (Gentoo 16.3 vanilla) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
```

Рис. 4.2.17 — Запуск lab09-3.asm

Как отмечалось в предыдущей лабораторной работе, при запуске программы аргументы командной строки загружаются в стек. Исследуем расположение аргументов командной строки в стеке после запуска программы с помощью gdb. Для начала установили точку останова перед первой инструкцией в программе и запустили ее (Рис. 4.2.18).



The screenshot shows a terminal window titled "Приложения Места" with the user "kstanchila@dk6n17 - lab09". The terminal displays the following text:

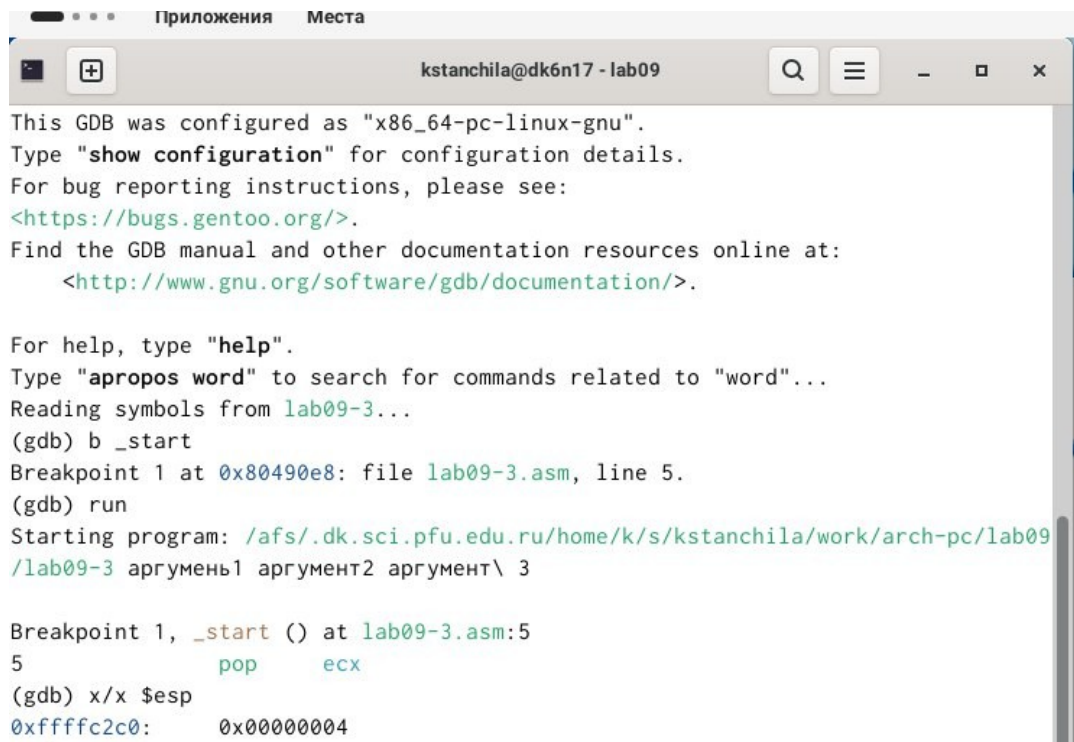
```
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 5.
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/k/s/kstanchila/work/arch-pc/lab09-3
/afs/.dk.sci.pfu.edu.ru/home/k/s/kstanchila/work/arch-pc/lab09-3 аргумент1 аргумент2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:5
5      pop     ecx
```

Рис. 4.2.18 — Установка точки останова

Адрес вершины стека храниться в регистре `esp` и по этому адресу располагается число равное количеству аргументов командной строки (включая имя программы) (Рис.4.2.19).



The screenshot shows the same terminal window as before, but now it displays the stack pointer value after the program has started. The terminal text is as follows:

```
Breakpoint 1, _start () at lab09-3.asm:5
5      pop     ecx
(gdb) x/x $esp
0xffffc2c0:      0x00000004
```

Рис. 4.2.19 — `esp`

Как видно, число аргументов равно 5 – это имя программы lab09-3 и непосредственно аргументы: аргумент1, аргумент, 2 и 'аргумент 3'. Посмотрели остальные позиции стека – по адресу [esp+4] располагается адрес в памяти где находится имя программы, по адресу [esp+8] храниться адрес первого аргумента, по адресу [esp+12] – второго и т. д. (Рис. 4.2.20).

```

This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://bugs.gentoo.org/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 5.
(gdb) run
Starting program: /afs/.dk.sci.pfu.edu.ru/home/k/s/kstanchila/work/arch-pc/lab09
/lab09-3 аргумент1 аргумент2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:5
5      pop     ecx
(gdb) x/x $esp
0xffffc2c0:  0x00000004
(gdb) x/x $esp+4
0xffffc2c4:  0xffffc517
(gdb) x/x $esp+8
0xffffc2c8:  0xffffc55e
(gdb) x/x $esp+12
0xffffc2cc:  0xffffc570
(gdb) x/x $esp+16
0xffffc2d0:  0xffffc582
(gdb) x/x $esp $esp+20
A syntax error in expression, near '$esp+20'.
(gdb) x/x $esp+20
0xffffc2d4:  0x00000000
(gdb) x/x $esp+24
0xffffc2d8:  0xffffc595
(gdb) █

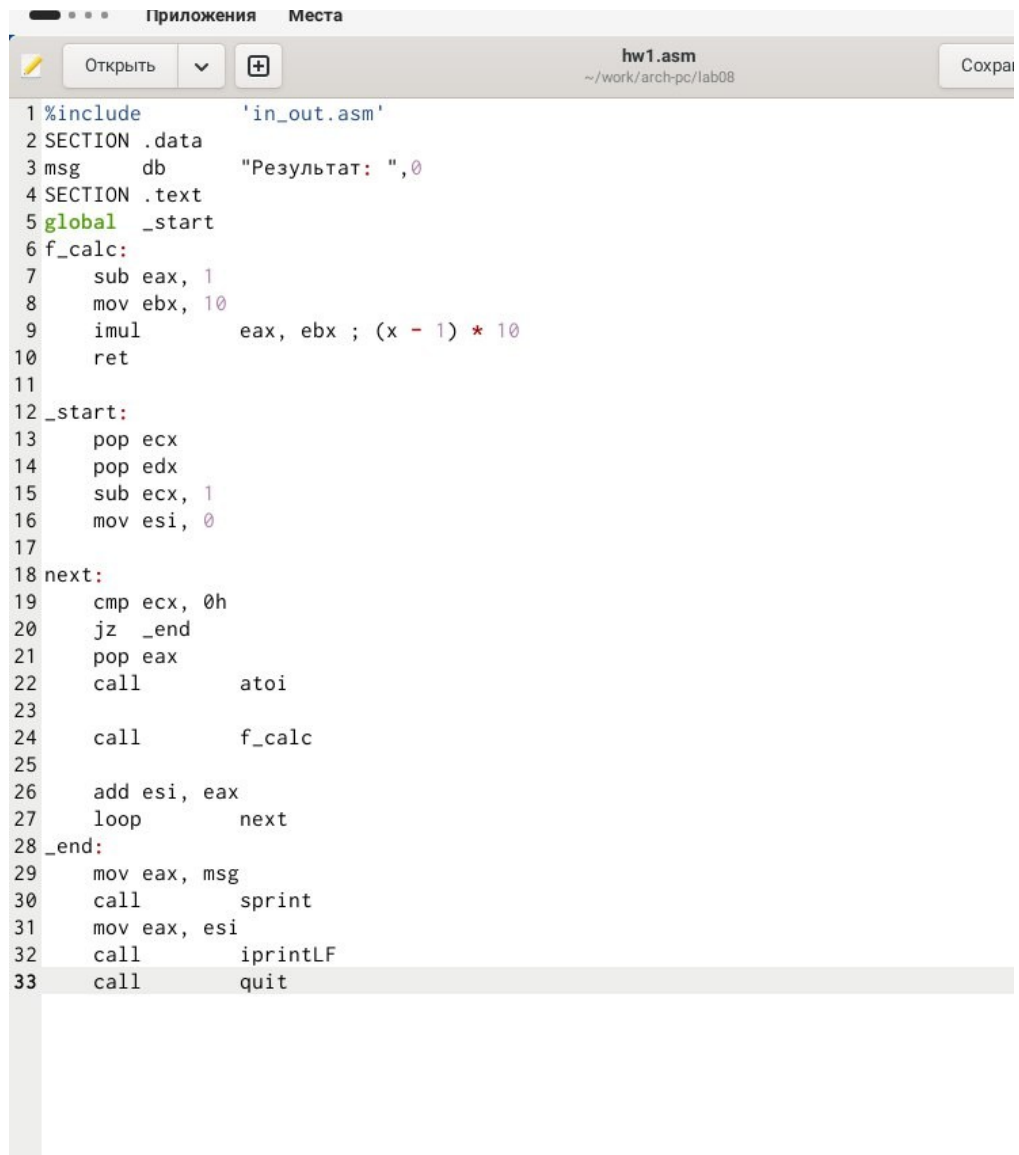
```

Рис. 4.2.20 — Остальные позиции стека

4.3 Задание для самостоятельной работы

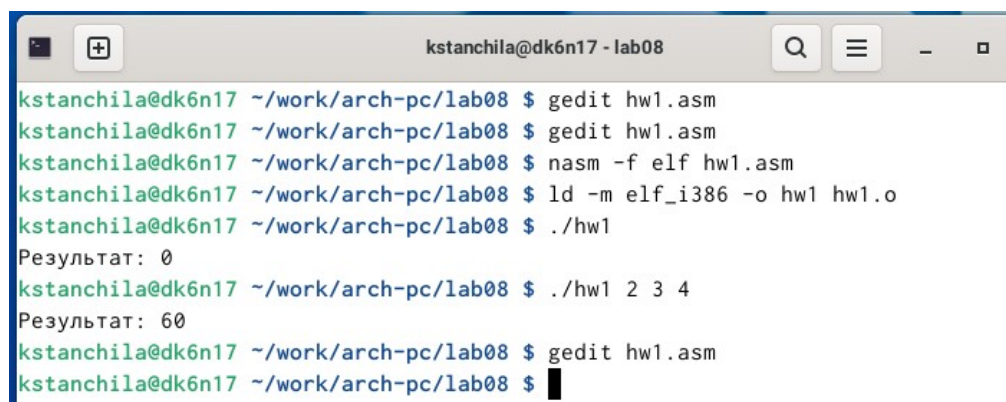
1. Преобразовала программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как

подпрограмму (Рис. 4.3.1 и Рис. 4.3.2).



```
1 %include 'in_out.asm'
2 SECTION .data
3 msg db "Результат: ",0
4 SECTION .text
5 global _start
6 f_calc:
7     sub eax, 1
8     mov ebx, 10
9     imul    eax, ebx ; (x - 1) * 10
10    ret
11
12 _start:
13     pop ecx
14     pop edx
15     sub ecx, 1
16     mov esi, 0
17
18 next:
19     cmp ecx, 0h
20     jz _end
21     pop eax
22     call    atoi
23
24     call    f_calc
25
26     add esi, eax
27     loop   next
28 _end:
29     mov eax, msg
30     call    sprint
31     mov eax, esi
32     call    iprintLF
33     call    quit
```

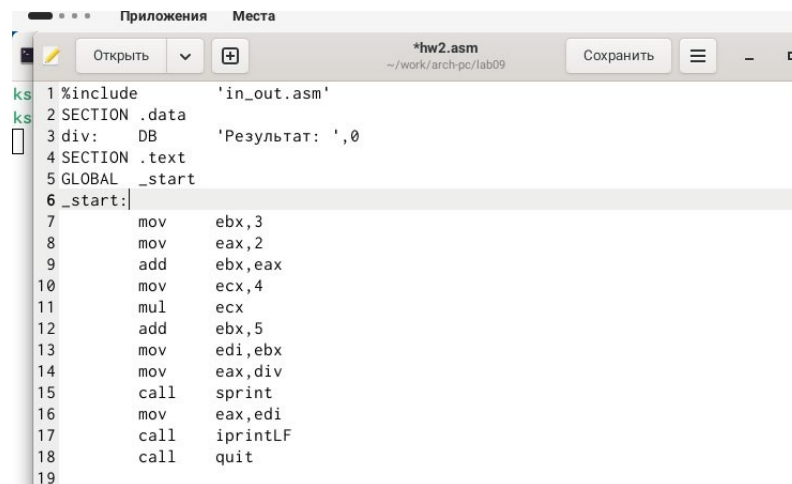
Рис. 4.3.1 — Текст самостоятельного задания №1



```
kstanchila@dk6n17 - lab08
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ nasm -f elf hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ld -m elf_i386 -o hw1 hw1.o
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./hw1
Результат: 0
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ ./hw1 2 3 4
Результат: 60
kstanchila@dk6n17 ~/work/arch-pc/lab08 $ gedit hw1.asm
kstanchila@dk6n17 ~/work/arch-pc/lab08 $
```

Рис. 4.3.2 — Результат выполнения самостоятельного задания №1

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверила это. С помощью отладчика GDB, анализируя изменения значений регистров, определила ошибку и исправила ее (Рис. 4.3.3 и Рис. 4.3.4).

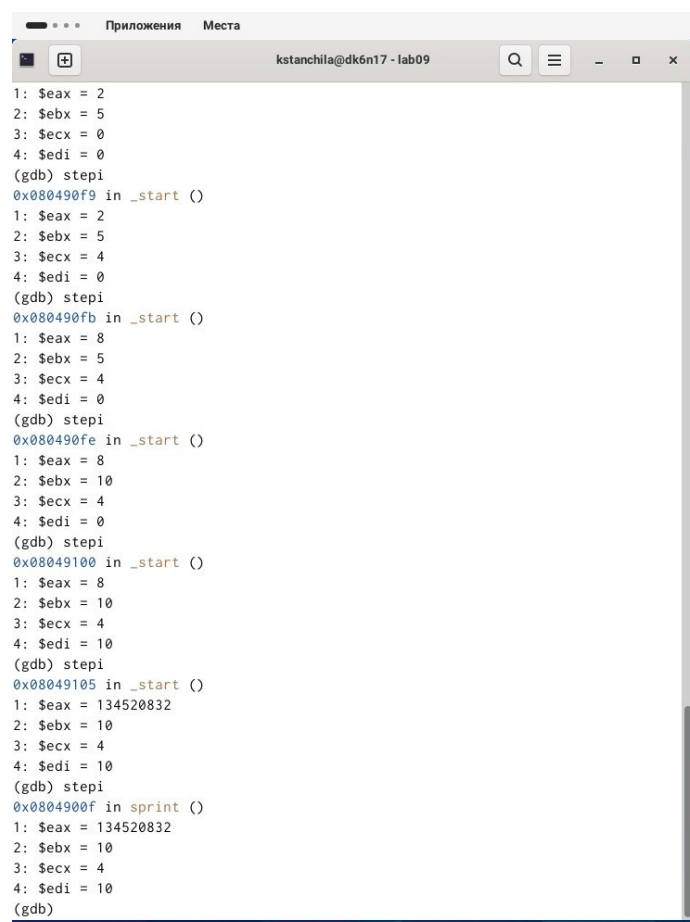


```

1 %include 'in_out.asm'
2 SECTION .data
3 div: DB 'Результат: ',0
4 SECTION .text
5 GLOBAL _start
6 _start:
7     mov     ebx,3
8     mov     eax,2
9     add     ebx,eax
10    mov     ecx,4
11    mul     ecx
12    add     ebx,5
13    mov     edi,ebx
14    mov     eax,div
15    call    sprintf
16    mov     eax,edi
17    call    iprintLF
18    call    quit
19

```

Рис. 4.3.3 — Текст самостоятельного задания №2



```

1: $eax = 2
2: $ebx = 5
3: $ecx = 0
4: $edi = 0
(gdb) stepi
0x080490f9 in _start ()
1: $eax = 2
2: $ebx = 5
3: $ecx = 4
4: $edi = 0
(gdb) stepi
0x080490fb in _start ()
1: $eax = 8
2: $ebx = 5
3: $ecx = 4
4: $edi = 0
(gdb) stepi
0x080490fe in _start ()
1: $eax = 8
2: $ebx = 10
3: $ecx = 4
4: $edi = 0
(gdb) stepi
0x08049100 in _start ()
1: $eax = 8
2: $ebx = 10
3: $ecx = 4
4: $edi = 10
(gdb) stepi
0x08049105 in _start ()
1: $eax = 134520832
2: $ebx = 10
3: $ecx = 4
4: $edi = 10
(gdb) stepi
0x0804900f in sprintf ()
1: $eax = 134520832
2: $ebx = 10
3: $ecx = 4
4: $edi = 10
(gdb)

```

Рис. 4.3.4 — Результат выполнения самостоятельного задания №2

5 Выводы

В результате выполнения данной лабораторной работы я приобрела навыки написания программ с использованием подпрограмм, ознакомилась с методами отладки при помощи GDB и его основными возможностями.

Список литературы

1. Курс на ТУИС
2. Лабораторная работа №9
3. Программирование на языке ассемблера NASM Столяров А. В